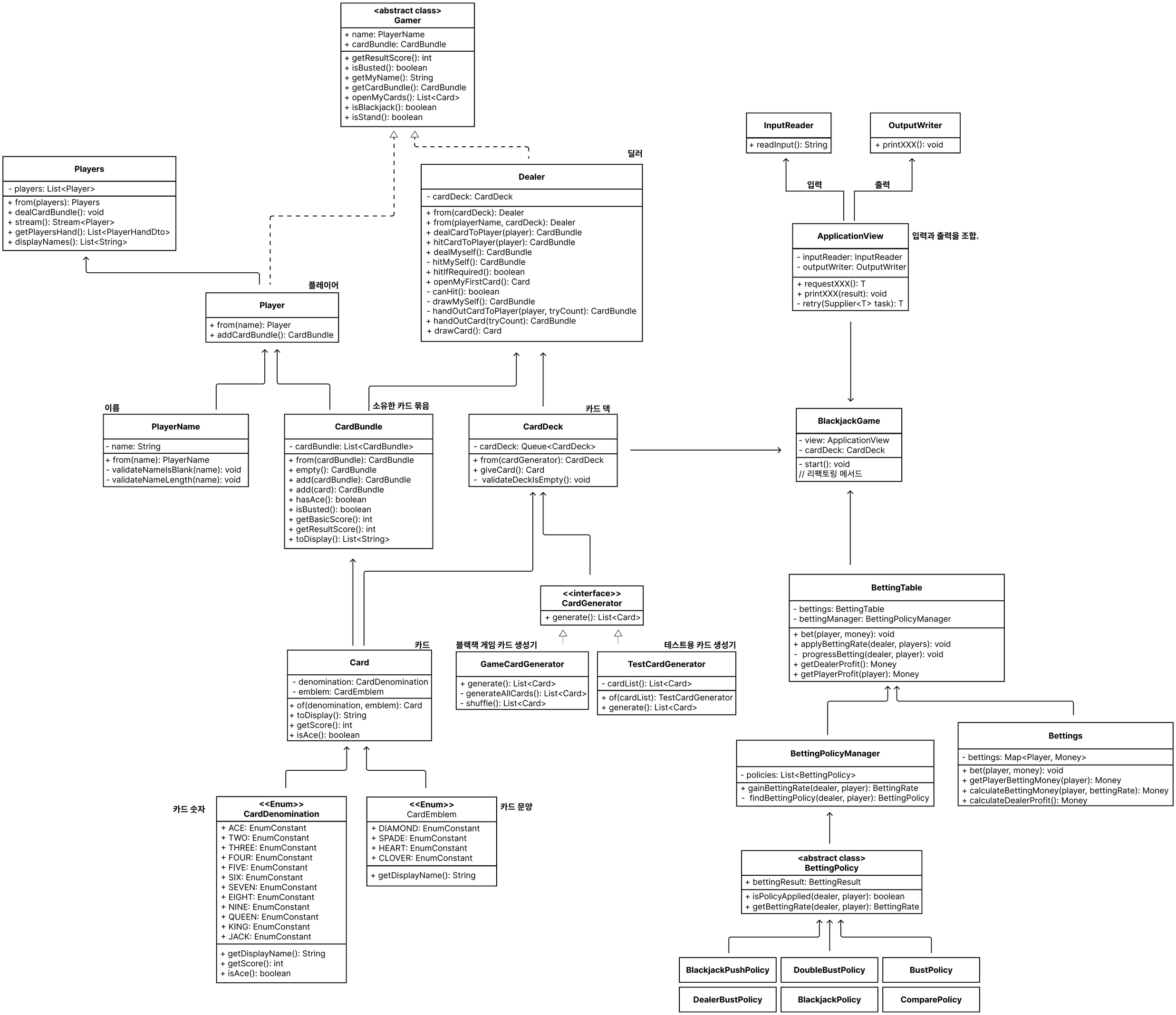
<abstract class>
Gamer
+ name: PlayerName
+ cardBundle: CardBundle
+ getResultScore(): int
+ isBusted(): boolean
+ getMyName(): String
+ getCardBundle(): CardBundle
+ openMyCards(): List<Card>
+ isBlackjack(): boolean
+ isStand(): boolean
Players
- players: List<Player>
+ from(players): Players
+ dealCardBundle(): void
+ stream(): Stream<Player>
+ getPlayersHand(): List<PlayerHandDto>
+ displayNames(): List<String>
딜러
Dealer
- cardDeck: CardDeck
+ from(cardDeck): Dealer
+ from(playerName, cardDeck): Dealer
+ dealCardToPlayer(player): CardBundle
+ hitCardToPlayer(player): CardBundle
+ dealMyself(): CardBundle
- hitMySelf(): CardBundle
+ hitIfRequired(): boolean
+ openMyFirstCard(): Card
- canHit(): boolean
- drawMySelf(): CardBundle
- handOutCardToPlayer(player, tryCount): CardBundle
+ handOutCard(tryCount): CardBundle
+ drawCard(): Card
플레이어
Player
+ from(name): Player
+ addCardBundle(): CardBundle
이름
PlayerName
- name: String
+ from(name): PlayerName
- validateNameIsBlank(name): void
- validateNameLength(name): void
소유한 카드 묶음
CardBundle
- cardBundle: List<CardBundle>
+ from(cardBundle): CardBundle
+ empty(): CardBundle
+ add(cardBundle): CardBundle
+ add(card): CardBundle
+ hasAce(): boolean
+ isBusted(): boolean
+ getBasicScore(): int
+ getResultScore(): int
+ toDisplay(): List<String>
카드 덱
CardDeck
- cardDeck: Queue<CardDeck>
+ from(cardGenerator): CardDeck
+ giveCard(): Card
- validateDeckIsEmpty(): void
<<interface>>
CardGenerator
+ generate(): List<Card>
블랙잭 게임 카드 생성기
GameCardGenerator
+ generate(): List<Card>
- generateAllCards(): List<Card>
- shuffle(): List<Card>
테스트용 카드 생성기
TestCardGenerator
- cardList(): List<Card>
+ of(cardList): TestCardGenerator
+ generate(): List<Card>
카드
Card
- denomination: CardDenomination
- emblem: CardEmblem
+ of(denomination, emblem): Card
+ toDisplay(): String
+ getScore(): int
+ isAce(): boolean
카드 숫자
<<Enum>>
CardDenomination
+ ACE: EnumConstant
+ TWO: EnumConstant
+ THREE: EnumConstant
+ FOUR: EnumConstant
+ FIVE: EnumConstant
+ SIX: EnumConstant
+ SEVEN: EnumConstant
+ EIGHT: EnumConstant
+ NINE: EnumConstant
+ QUEEN: EnumConstant
+ KING: EnumConstant
+ JACK: EnumConstant
+ getDisplayName(): String
+ getScore(): int
+ isAce(): boolean
<<Enum>>
CardEmblem
+ DIAMOND: EnumConstant
+ SPADE: EnumConstant
+ HEART: EnumConstant
+ CLOVER: EnumConstant
+ getDisplayName(): String
카드 문양
InputReader
+ readInput(): String
OutputWriter
+ printXXX(): void
입력
출력
ApplicationView
- inputReader: InputReader
- outputWriter: OutputWriter
+ requestXXX(): T
+ printXXX(result): void
- retry(Supplier<T> task): T
입력과 출력을 조합.
BlackjackGame
- view: ApplicationView
- cardDeck: CardDeck
- start(): void
// 리팩토링 메서드
BettingTable
- bettings: BettingTable
- bettingManager: BettingPolicyManager
+ bet(player, money): void
+ applyBettingRate(dealer, players): void
- progressBetting(dealer, player): void
+ getDealerProfit(): Money
+ getPlayerProfit(player): Money
BettingPolicyManager
- policies: List<BettingPolicy>
+ gainBettingRate(dealer, player): BettingRate
- findBettingPolicy(dealer, player): BettingPolicy
Bettings
- bettings: Map<Player, Money>
+ bet(player, money): void
+ getPlayerBettingMoney(player): Money
+ calculateBettingMoney(player, bettingRate): Money
+ calculateDealerProfit(): Money
<abstract class>
BettingPolicy
+ bettingResult: BettingResult
+ isPolicyApplied(dealer, player): boolean
+ getBettingRate(dealer, player): BettingRate
BlackjackPushPolicy
DoubleBustPolicy
BustPolicy
DealerBustPolicy
BlackjackPolicy
ComparePolicy